PS-7

Arijit Das

2026-03-24

Comparison of least squares, ridge regression, LASSO and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
rm(list=ls())
library(ISLR)

df=na.omit(Hitters)
dim(df)

## [1] 263  20
```

The Hitters dataset contains $n = 263$ observations and $p = 19$ predictor variables. Let Y_i denote the response variable (Salary) for the ith player and let X_{ij} denote the jth predictor for the ith player, where $j = 1,2, \dots ,19$.

The multiple linear regression model is given by

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \beta_3 X_{i3} + \cdots + \beta_{19} X_{i19} + \varepsilon_i, \quad i = 1,2, \dots ,263$$

where

$$\varepsilon_i \sim N(0, \sigma^2)$$

Here,

- Y_i : Salary of the ith player
- X_{ij} : j^{th} predictor variable for the ith player
- β_0 : Intercept
- β_j : Regression coefficient (unknown) corresponding to the jth predictor for $j = 1,2, \dots ,19$
- ε_i : Random error term

i=1(1)263 and j=1(1)19

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
set.seed(123)
test.id=sample(1:263,100)
train.data=df[-test.id,]
```

```
test.data=df[test.id,]
dim(train.data)

## [1] 163  20

dim(test.data)

## [1] 100  20
```

There are 263 rows, out of which we choose 100 to form the Test data. And the remaining 163 are put together to form the Training data.

2. Train the model using least squares and report the train MSE and test MSE.

```
m1=lm(Salary~.,data=train.data)
mse.train=1/163*(sum((train.data$Salary-predict(m1))^2))
y.hat.test=predict(m1,newdata=test.data)
mse.test=mean((test.data$Salary-y.hat.test)^2)
list(Training_MSE=mse.train,Test_MSE=mse.test)

## $Training_MSE
## [1] 105334.2
##
## $Test_MSE
## [1] 87803.91
```

Training MSE = 105334.2 Test MSE = 87803.91

The training MSE represents the average squared difference between the observed salaries and the predicted salaries for the training data. The test MSE measures the average squared prediction error on the test data.

Since the test MSE is slightly smaller than the training MSE, the model does not show evidence of overfitting. It suggests that the least squares regression model generalizes reasonably well to unseen data.

3. Fit ridge regression models on the training set, with grid values of λ.

```
library(Matrix)
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.3.3

## Loaded glmnet 4.1-8

lambda.grid = 10^seq(3, -3, length = 100)
x=model.matrix(Salary~.,train.data)[,-1]
x.test=model.matrix(Salary~.,test.data)[,-1]
ridge.model=glmnet(x, train.data$Salary, alpha = 0,lambda=lambda.grid)
```

The ridge regression minimizes the penalized residual sum of squares (PRSS):

$$PRSS = RSS + \lambda \sum_{j=1}^{19} \beta_j^2$$

where

$$RSS = \sum_{i=1}^{263} \left(Y_i - \beta_0 - \sum_{j=1}^{19} \beta_j X_{ij} \right)^2$$

where $\lambda > 0$ is the tuning parameter controlling the amount of shrinkage applied to the regression coefficients.

We take 100 grid values of λ from 10^{-3} to 10^{3}. Report the following:

(i) the dimension of the coefficient matrix so obtained.

```
dim(coef(ridge.model))

## [1]  20 100
```

Thus, the ridge model estimates 20 coefficients (19 predictors, 1 intercept) for each of the 100 values of λ, producing a 20×100 coefficient matrix.

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. Comment on the output.

```
beta50=coef(ridge.model)[,50]
beta60=coef(ridge.model)[,60]

l2.50=sqrt(sum(beta50^2))
l2.60=sqrt(sum(beta60^2))

list(l2.loss_lambda50=l2.50,l2.loss_lambda60=l2.60)

## $l2.loss_lambda50
## [1] 261.6611
##
## $l2.loss_lambda60
## [1] 263.8089
```

The L_2 norm measures the overall magnitude of the coefficient vector. The slightly larger value for the 60th λ indicates a slightly larger overall coefficient magnitude compared to the 50th λ. However, the difference is small, suggesting that the level of shrinkage applied to the coefficients is very similar for these two values of λ.

(iii) Test MSE corresponding to these two choices of λ.

```
pred50 = predict(ridge.model, s = lambda.grid[50], newx = x.test)
pred60 = predict(ridge.model, s = lambda.grid[60], newx = x.test)
```

```r
mse50 = mean((test.data$Salary - pred50)^2)
mse60 = mean((test.data$Salary - pred60)^2)

list(mse_lambda50=mse50,mse_lambda60=mse60)
```

```
## $mse_lambda50
## [1] 86296.03
##
## $mse_lambda60
## [1] 86486.01
```

Final Result

```r
out=data.frame(Sl=c(50,60),Lambda=c(lambda.grid[50],lambda.grid[60]),L2_Loss=
c(l2.50,l2.60),Test_MSE=c(mse50,mse60))
out
```

```
##   Sl    Lambda  L2_Loss Test_MSE
## 1 50 1.0722672 261.6611 86296.03
## 2 60 0.2656088 263.8089 86486.01
```

Since the second λ is smaller, the penalty is weaker, leading to a slightly larger L_2 norm (less shrinkage of coefficients). However, the test MSE is slightly higher, indicating marginally worse predictive performance.

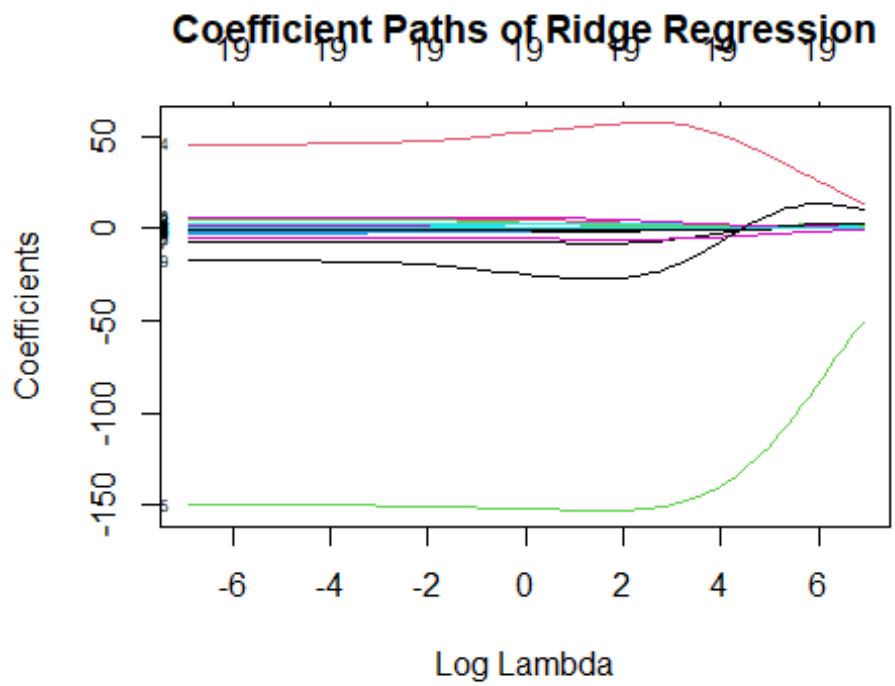
Thus, the 50th λ provides slightly better prediction accuracy due to stronger tuning.

Conclusion:

The grid values chosen here do not show a strong difference in coefficient shrinkage or prediction error. Both λ values lead to similar L_2 norms and test MSE, suggesting that they fall in a stable region of the ridge solution path. A wider grid of λ values would better illustrate the impact of tuning.

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of λ chosen by you. Interpret the graph.

```r
model = glmnet(x, train.data$Salary,
                   alpha = 0,
                   lambda = lambda.grid,
                   standardize = TRUE)
plot(model, xvar = "lambda", label = TRUE, main ="Coefficient Paths of Ridge
Regression")
```



Comment:

The plot shows standardized ridge regression coefficients against −log(λ). As λ increases (moving left), all coefficients shrink smoothly toward zero due to the L2 penalty.

No coefficient becomes exactly zero, indicating that ridge regression does not perform variable selection. A few coefficients remain relatively large, suggesting stronger predictors, while most stay close to zero.

For small λ (right side), coefficients are larger and resemble OLS estimates, whereas for large λ (left side), they are more stable but heavily shrunk, reflecting the bias–variance trade-off.

5. How would the graph in (4) change if instead of λ you plot on the x axis the ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient.

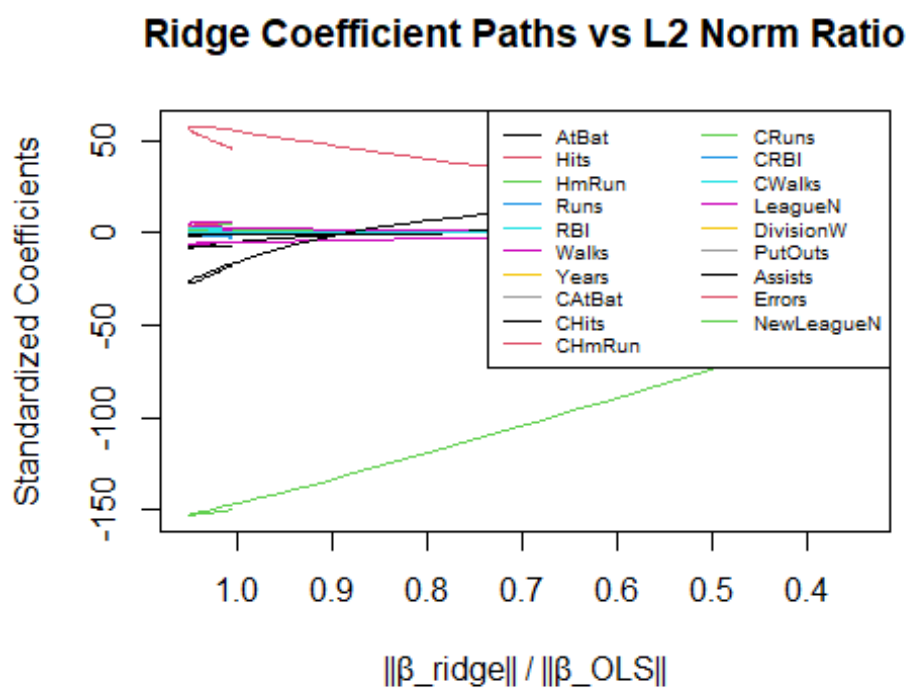
```
ols.mod = lm(Salary ~ ., data = train.data)
beta.ols = coef(ols.mod)[-1]

beta.ridge = as.matrix(coef(ridge.model))[-1, ]

# L2 norms
ridge.norm = apply(beta.ridge, 2, function(b) sqrt(sum(b^2)))
ols.norm = sqrt(sum(beta.ols^2))
```

```r
#Ratio
ratio = ridge.norm / ols.norm

#Plot with reversed x-axis
matplot(ratio, t(beta.ridge),type="l", lty=1,xlab="||β_ridge|| /
||β_OLS||",ylab="Standardized Coefficients",main="Ridge Coefficient Paths vs
L2 Norm Ratio",xlim=rev(range(ratio)))
legend("topright",legend = rownames(beta.ridge),col = 1:nrow(beta.ridge),lty
= 1,cex = 0.6,ncol = 2)
```



Comment:

The plot shows ridge regression coefficient paths against the ratio of l2 norm of the ridge regression coefficient, which measures the degree of shrinkage.

As the ratio decreases from 1 to 0, all coefficients shrink smoothly toward zero due to increasing regularization. None of the coefficients become exactly zero, confirming that ridge regression does not perform variable selection.

A few predictors retain relatively large magnitudes across the range, indicating stronger influence on Salary, while most coefficients remain close to zero, suggesting weaker effects.

6. Run the Ridge regression on training data by choosing λ by cross-validation.Find the test MSE for the optimal choice of λ obtained using cross validation.

```
set.seed(123)
cv.ridge=cv.glmnet(x,train.data$Salary,alpha=0)

best.lambda=cv.ridge$lambda.min
best.lambda

## [1] 404.3208

ridge.pred=predict(cv.ridge,s=best.lambda,newx=x.test)

#Test MSE
mse.cv=mean((test.data$Salary-ridge.pred)^2)
mse.cv

## [1] 93690.09
```

Comment:

The test MSE obtained using the optimal λ from cross-validation is 93690.09.

This value represents the average squared prediction error of the ridge regression model on unseen test data. It indicates the model's predictive accuracy after applying regularization.

Compared to the least squares test MSE (87803.91), the ridge regression model has a slightly higher test error. This suggests that, for this dataset, regularization does not improve predictive performance and the least squares model performs marginally better.

7. Fit a LASSO model on the training set, with λ chosen by cross-validation. Report the test MSE, along with the number of non-zero coefficient estimates.

```
set.seed(123)
cv.lasso = cv.glmnet(x, train.data$Salary, alpha = 1)
best.lambda.lasso = cv.lasso$lambda.min
best.lambda.lasso

## [1] 16.70659

lasso.pred = predict(cv.lasso, s = best.lambda.lasso, newx = x.test)
mse.lasso = mean((test.data$Salary - lasso.pred)^2)
#Test MSE
mse.lasso

## [1] 93914.89

#Number of non-zero coefficients (excluding intercept)
lasso.coef = coef(cv.lasso, s = best.lambda.lasso)
nonzero = sum(lasso.coef[-1] != 0)
nonzero
```

```
## [1] 8
```

Comment:

The test MSE of the LASSO model is 93914.89, which represents the prediction error on unseen data using the optimal λ selected by cross-validation.

The model retains 8 non-zero coefficients, indicating that LASSO has performed variable selection by shrinking the remaining coefficients exactly to zero.

This suggests that a simpler model with only 8 predictors is sufficient to explain most of the variation in Salary. However, the test MSE is slightly higher than that of the least squares model, indicating a small loss in predictive accuracy in exchange for a more interpretable and sparse model.

8. Fit an Elastic Net model on the training set, with λ1 and λ2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```r
set.seed(123)
alpha.grid = seq(0, 1, by = 0.1)
cv.errors = rep(NA, length(alpha.grid))
cv.models = list()
for(i in 1:length(alpha.grid)){
  cv.models[[i]] = cv.glmnet(x, train.data$Salary, alpha = alpha.grid[i])
  cv.errors[i] = min(cv.models[[i]]$cvm)
}

#Best alpha and model
best.index = which.min(cv.errors)
best.alpha = alpha.grid[best.index]
best.model = cv.models[[best.index]]

#Best lambda
best.lambda = best.model$lambda.min

#Convert to lambda1 and lambda2
lambda1 = best.lambda * (1 - best.alpha)
lambda2 = best.lambda * best.alpha

lambda1

## [1] 86.0411

lambda2

## [1] 9.560123
```

```r
# Predictions
elastic.pred = predict(best.model, s = best.lambda, newx = x.test)

#Test MSE
mse.elastic = mean((test.data$Salary - elastic.pred)^2)
mse.elastic

## [1] 91957.09

# Non-zero coefficients (excluding intercept)
elastic.coef = coef(best.model, s = best.lambda)
nonzero.elastic = rownames(elastic.coef)[elastic.coef[,1] != 0][-1]
nonzero.elastic

##  [1] "Hits"      "HmRun"     "RBI"       "Walks"     "CAtBat"    "CHits"
##  [7] "CHmRun"    "CRuns"     "CRBI"      "LeagueN"   "DivisionW" "PutOuts"
## [13] "Assists"   "Errors"
```

Comment:

The test MSE of the Elastic Net model is 91957.09, which indicates its prediction error on unseen data using the optimal tuning parameters selected via cross-validation.

The model retains several predictors with non-zero coefficients, including Hits, HmRun, RBI, Walks, CAtBat, CHits, CHmRun, CRuns, CRBI, LeagueN, DivisionW, PutOuts, Assists, and Errors. This shows that Elastic Net performs both shrinkage and variable selection, retaining more variables than LASSO but fewer than ridge regression.

Overall, the model achieves a balance between complexity and interpretability, with moderate predictive performance compared to ridge and least squares.

9. Compare the test MSE 's of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

The test MSE values obtained are as follows: Least Squares (87803.91), Ridge Regression (93690.09), LASSO (93914.89), and Elastic Net (91957.09).

Among all the models, the least squares regression yields the lowest test MSE, indicating the best predictive performance for this dataset. Ridge, LASSO, and Elastic Net introduce regularization, which slightly increases the test error in this case.

This suggests that the data does not suffer significantly from overfitting or multicollinearity, so regularization does not provide improvement in prediction accuracy. However, LASSO and Elastic Net produce simpler and more interpretable models by selecting a subset of important predictors.

Overall, the salary of a baseball player can be predicted reasonably well using these models, with least squares performing the best in terms of accuracy, while LASSO and Elastic Net offer a trade-off between accuracy and model simplicity.